builders

PlayerConfiguration
- playerName: String
- playerColour: PlayerColour

+ getPlayerName(): String
+ getPlayerColour(): PlayerColour

<<enumeration>> PlayerColour
RED
BLUE
GREEN
YELLOW
BLACK

GameConfiguration
- playerConfigList: List<PlayerConfiguration>
- map: MapType

+ getPlayerConfigs(): List<PlayerConfiguration>
+ getMapType(): MapType

<<enumeration>> MapType
LONDON

<<interface>> GameBuilder
+ buildBoard(): void
+ buildDecks(): void
+ buildPlayers(playerConfigs: List<PlayerConfiguration>): void
+ buildServices(): void
+ dealStartingHands(): void
+ getResult(): Game

LondonGameBuilder
- newGame: Game

GameSetupDirector
+ constructGame(config: GameConfiguration): Game

main

<<interface>> PendingDecisionVisitor
+ visitTicketDecision(decision: TicketDecision): void

GameController
- game: Game
+ onStartGame(config: GameConfiguration): void
+ onDrawTrainCard(slot: int): void
+ onClaimRoute(route: Route, cards: List<TrainCard>): void
+ onDrawDestinationCards(): void
+ finalizeDestinationCardDecision(keep: List<DestinationCard>, discard: List<DestinationCard>): void
- processAndRoute(action: ActionCommand): void

Game
- players: List<Player>
- currentPlayerIndex: int
- board: Board
- scoringService: ScoringService
- turnMediator: TurnMediator
- lastRoundTracker: LastRoundTracker
- isSetupPhase: boolean
- trainCardDeck: TrainCardDeck
- destinationCardDeck: DestinationCardDeck

+ playAction(action: ActionCommand): ActionResult
+ endTurn(): EndTurnResult
+ getCurrentPlayer(): Player
+ isGameOver(): boolean
+ resolveGameOver(): EndTurnResult
+ getTrainCardDeck(): TrainCardDeck
+ getDestinationCardDeck(): DestinationCardDeck
+ startSetupPhase(): void
+ isSetupPhase(): boolean
- prepareTicketsForCurrentSetupPlayer(): void
- advanceToNextPlayer(): void

deck

<<abstract>> Deck
- inDeck: List<Card>

+ draw(): Card
+ shuffle(): void
+ getInDeck(): List<Card>
+ isInDeckEmpty(): boolean

DestinationCardDeck
+ drawThree(): List<DestinationCard>
+ returnToBottom(cards: List<DestinationCard>): void

TrainCardDeck
- discardedPile: List<TrainCard>
- faceUpPile: List<TrainCard>

+ draw(slot: int): TrainCard
+ drawInDeck(): TrainCard
+ isFaceUpEmpty(): boolean
+ getFaceUpCards(): List<TrainCard>
+ discard(cards: List<TrainCard>): void
+ canDrawFaceUp(slot: int): boolean
- checkAndReplaceLocomotives(): void
- shuffleDiscardIntoDeck(): void
- replaceFaceUp(slot: int): void

card

<<abstract>> Card
- id: int

+ getId(): int

TrainCard
- colour: Colour {readOnly}

+ getColour(): Colour
+ isLocomotive(): boolean

DestinationCard
- from: City
- to: City
- points: int
- isCompleted: boolean

+ getFrom(): City
+ getTo(): City
+ getPoints(): int
+ isCompleted(): boolean
+ complete(): void

map

Board
- cities: List<City>
- routes: List<Route>

+ getRoute(from: City, to: City): Route
+ isPathConnected(player: Player, from: City, to: City): boolean
+ getAvailableRoutes(): List<Route>

Route
- cityA: City
- cityB: City
- length: int
- colour: Colour
- isClaimed: boolean
- owner: Player
- claimStrategy: RouteClaimStrategy

+ getCityA(): City
+ getCityB(): City
+ getLength(): int
+ getColour(): Colour
+ isClaimed(): boolean
+ setOwner(player: Player): void
+ getOwner(): Player
+ getClaimStrategy(): RouteClaimStrategy

<<enumeration>> Colour
RED
ORANGE
YELLOW
GREEN
BLUE
PINK
WHITE
BLANK
GREY
WILD

City
- name: String
- x: int
- y: int

+ getName(): String
+ getX(): int
+ getY(): int

scoring

ScoringService
- routeScoring: RouteScoringStrategy
- endGameScoring: List<EndGameScoringStrategy>

+ scoreRoute(player: Player, route: Route): int
+ scoreEndGame(players: List<Player>, board: Board): void

<<interface>> RouteScoringStrategy
+ score(route: Route): int

<<interface>> EndGameScoringStrategy
+ apply(players: List<Player>, board: Board): void

StandardRouteScoringRule
- scoreTable: Map<Integer, Integer> {readOnly}

DestinationTicketScoringRule

LongestRouteScoringRule
- bonusPoints: int = 10 {readOnly}

+ computeLongestPath(player: Player): int

character

Player
- name: String
- colour: PlayerColour
- remainingTrainPieces: int
- score: int
- hand: PlayerHand

+ getHand(): PlayerHand
+ getScore(): int
+ getRemainingTrainPieces(): int
+ getName(): String
+ getColour(): PlayerColour
+ addScore(points: int): void
+ hasEnoughTrainPieces(count: int): boolean
+ reduceTrainPieces(numOfPieces: int): void

PlayerHand
- trainCards: Map<Colour, List<TrainCard>>
- destinationCards: List<DestinationCard>

+ getTrainCards(): Map<Colour, List<TrainCard>>
+ getDestinationCards(): List<DestinationCard>
+ getLocomotiveCount(): int
+ getTrainCardCountByColour(colour: Colour): int
+ addTrainCard(card: TrainCard): void
+ addTrainCards(cards: List<TrainCard>): void
+ addDestinationCards(cards: List<DestinationCard>): void
+ removeTrainCards(cards: List<TrainCard>): void
+ removeDestinationCard(cards: List<DestinationCard>): void
+ hasEnoughOfAnyColour(needed: int): boolean

strategy

<<interface>> RouteClaimStrategy
+ validate(player: Player, route: Route, spentCards: List<TrainCard>): boolean
+ applyPayment(player: Player, route: Route, spentCards: List<TrainCard>, trainCardDeck: TrainCardDeck): void

GrayRouteClaimStrategy

StandardRouteClaimStrategy

turnManager

LastRoundTracker
- triggered: boolean
- turnsRemaining: int
- triggeringPlayer: Player

+ checkAndTrigger(player: Player, totalPlayers: int): boolean
+ decrementRemainingTurns(): void
+ isTriggered(): boolean
+ isOver(): boolean
+ getTriggeringPlayer(): Player

TurnMediator
- remainingActionCost: int
- pendingDecision: PendingDecision

+ isTurnComplete(): boolean
+ reset(): void
+ processAction(action: ActionCommand): ActionResult
+ getPendingDecision(): PendingDecision
+ clearPendingDecision(): void
+ setPendingDecision(): void

command

<<enumeration>> ActionStatus
SUCCESS,
FAILED,
PENDING

ActionResult
- status: ActionStatus
- message: String
- endTurnResult: EndTurnResult
- pendingDecision: PendingDecision

+ isSuccess(): boolean
+ getEndTurnResult(): EndTurnResult
+ static success(message: String): ActionResult
+ static fail(message: String): ActionResult
+ static pending(decision: PendingDecision): ActionResult
+ setEndTurnResult(EndTurnResult): void
+ getPendingDecision(): PendingDecision
+ hasPendingDecision(): boolean

<<interface>> PendingDecision
+ accept(visitor: PendingDecisionVisitor): void

TicketDecision
- drawnCards: List<DestinationCard>
- minKeep: int

+ getCards(): List<DestinationCard>

<<interface>> ActionCommand
+ execute(): ActionResult
+ getCost(): ActionCost

<<enumeration>> ActionCost
FREE
STANDARD,
FULL
+ points: int

+ getPoints(): int

EndTurnResult
- outcome: EndTurnOutcome
- nextPlayer: Player
- finalScores: Map<Player, Integer>

+ static nextTurn(nextPlayer: Player): EndTurnResult
+ static lastRoundTriggered(nextPlayer: Player): EndTurnResult
+ static gameOver(scores: Map<Player, Integer>): EndTurnResult
+ getOutcome(): EndTurnOutcome
+ static lastRoundContinues(nextPlayer: Player): EndTurnResult

<<enumeration>> EndTurnOutcome
NEXT_PLAYER_TURN
LAST_ROUND_TRIGGERED
LAST_ROUND_CONTINUES
GAME_OVER

DrawDestCommand
- currentPlayer: Player
- ticketDeck: DestinationCardDeck

ClaimRouteCommand
- route: Route
- trainCardList: List<TrainCard>
- currentPlayer: Player
- board: Board

DrawTrainCardCommand
- slot: int
- currentPlayer: Player
- trainCardDeck: TrainCardDeck

ResolveDrawDestCommand
- keepDestCards: List<DestinationCard>
- discardDestCards: List<DestinationCard>
- currentPlayer: Player
- ticketDeck: DestinationCardDeck

performed by